UTM
UNIVERSITI TEKNOLOGI MALAYSIA

DATA STRUCTURE AND ALGORITHM SECJ2013

MINI PROJECT : AYAM GUNTING VIRAL ORDERING SYSTEM USING QUEUE

DUE DATE:
16 January 2025

GROUP:
8

NAME	MATRIC NO
DAMIYA AINA BINTI BASIR ABD SHAMMAD	A23CS0220
LAM YOKE YU	A23CS0233
MUHAMMAD HAFIZI SYAMIR BIN HERMAN	A22EC0213
TAN YI YA	A23CS0187

Application of Stack in a Real-World Scenario: Ayam Gunting Viral Ordering System Using Queue

1. Case Study Documentation

1.1 Synopsis

Ayam Gunting Viral Ordering System is a Queue based system for managing orders of a small scale *ayam gunting* (fried chicken pieces) business. Workers that produce delicious *ayam gunting* can use this system to keep track of the waiting list of orders with the order ID as each entry of order and the quantities ordered. This system will demonstrate the real world application of queue data structures, where orders are handled in a First In First Out (FIFO) manner.

1.2 Objective

1.2.1 To demonstrate the use of a queue to manage orders in a sequential, FIFO manner.

1.2.2 To add orders to the queue with unique order IDs and quantities.

1.2.3 To view all current orders in the queue with details such as order ID and quantity.

1.2.4 To delete specific orders using the order ID (for cancel order)

1.2.5 To view the next order pending to be processed.

1.2.6 To remove the front order from the pending list when it is ready, signifying the order as being fulfilled.

1.3 System Requirement & Analysis

Most ayam gunting shops use paper to record the number of orders, while some others use their own memory to remember the order. Consequently, the order can be mixed up and not following the order, or it may be forgotten by the vendor.

Therefore, a system is needed to keep track of the order. The Ayam Gunting Ordering System is proposed to solve the problem. A queue data structure works that

implements First In First Out (FIFO) perfectly in this scenario, where the first people who make the order will get their other first.

Several functions are implemented including `addOrder()`, `cancelOrder()`, `orderReady()`, `displayOrder()` and `getFront()`.

The `addOrder()` function, as its name suggests, adds a new order to the queue. It gets the user input of the quantity of ayam gunting for the order, and gives the user a unique order id.

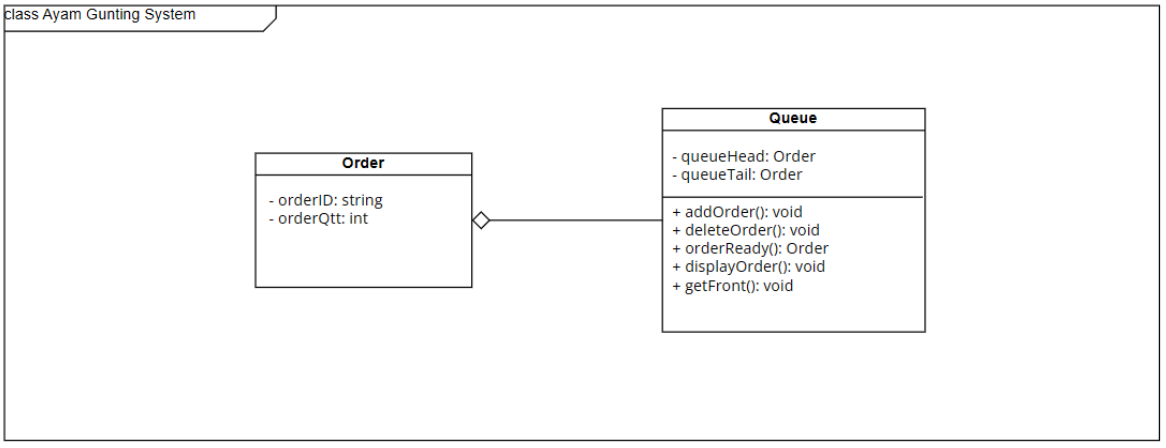
The `cancelOrder()` function on the other hand, cancels the order based on the orderID that is inputted by the user. This increases the flexibility of the system.

The `orderReady()` function would remove the first node of the queue, as the first order is ready.

`displayOrder()`, as its name suggests, displays all the orders in the queue.

The `getFront()` function gets the first order in the queue. This function allows the vendor to know the quantity of Ayam Gunting in the order that it is currently preparing.

1.4 Class Diagram



2. Code Implementation

```
#include <iostream>
using namespace std;

int orderCount = 0;

class Order {
public:
    int orderID;
    int orderQtt;
    Order* next;
};

class Queue {
public:
    Order* queueHead;
    Order* queueTail;

    Queue(){
        queueHead = nullptr;
        queueTail = nullptr;
    }

    bool isEmpty() {
        return queueHead == nullptr;
    }

    void addOrder() { // HAFIZI'S PART
        int quantity;
        cout << "Enter Order Quantity: ";
        cin >> quantity;

        // Create a new order

        Order* newOrder = new Order();
        newOrder->orderID = ++orderCount;
        newOrder->orderQtt = quantity;
        newOrder->next = nullptr;

        // If the queue is empty, the new order becomes the head and
tail

        if (isEmpty()) {
```

```
        queueHead = newOrder;
        queueTail = newOrder;
    } else {
        // Add the new order to the end of the queue
        queueTail->next = newOrder;
        queueTail = newOrder;
    }

    cout << "Your order ID is " << queueTail->orderID << endl;
    cout << "Order has been added to the queue.\n\n";
}

void getFront() { // HAFIZI'S PART
    if (isEmpty()) {
        cout << "No orders in the queue.\n";

    } else {

        cout << "Front Order Details: \n";
        cout << "Order ID: " << queueHead->orderID
             << ", Quantity: " << queueHead->orderQtt << endl;

    }

    cout << "\n";
}

void cancelOrder() { // LAM YOKE KU'S PART
    if (isEmpty()) {
        cout << "There is no order.\n\n";
        return;
    }

    Order* current = queueHead;
    Order* prev = nullptr;
    int id;

    cout << "Order ID: ";
    cin >> id;

    while (current) {
        // If order found
        if (current->orderID == id) {
```

```
            // If order is the only one in the queue
            if (prev == nullptr && current->next == nullptr) {
                queueHead = nullptr;
                queueTail = nullptr;
            }
            // If order is the head of the queue
            else if (current == queueHead) {
                queueHead = queueHead->next;
            }
            // If order is the tail of the queue
            else if (current == queueTail) {
                queueTail = prev;
                queueTail->next = nullptr;
            }
            // If order is in the middle of the queue
            else {
                prev->next = current->next;
            }

            cout << "Order Found.\n"
                 << "Order ID: " << current->orderID << endl
                 << "Order Quantity: " << current->orderQtt << endl
                 << "Cancelling Order...\n\n";

            delete current;
            return;
        } else {
            prev = current;
            current = current->next;
        }
    }

    cout << "Order Not Found.\n\n";
}
```

```
//display all orders in queue edited
    void displayOrders() { // DAMIYA'S PART
        if (!queueHead) {
            cout << "No orders in the queue.\n";
            return;
        }

        Order* temp = queueHead;
        cout << "Current orders in the queue:\n";
        while (temp) {
            cout << "Order ID: " << temp->orderID
                 << ", Quantity: " << temp->orderQtt << endl;
            temp = temp->next;
        }
        cout << "\n";
    }

    // Notify and remove the front order when it's ready (edited)
    void orderReady() { // DAMIYA'S PART
        if (!queueHead) {
            cout << "No orders to process.\n";
            return;
        }

        // Notify about the order that is ready
        cout << "Order ready: Order ID: " << queueHead->orderID
             << ", Quantity: " << queueHead->orderQtt << endl;

        // Delete the front order
        Order* temp = queueHead;
        queueHead = queueHead->next;

        if (!queueHead) {
            queueTail = nullptr;
        }

        delete temp;
        cout << "\n";
    }


};
```

```cpp
int main() { //TAN YI YA'S PART
    Queue q;
    int choice;

    while (true) {
        cout << "<<<<< Ayam Gunting Viral Ordering System >>>>>\n";
        cout << "1. New Order\n";
        cout << "2. Cancel Order\n";
        cout << "3. Order Ready\n";
        cout << "4. Display Order\n";
        cout << "5. Get First Order\n";
        cout << "6. Exit\n";
        cout << "Enter your choice: ";
        cin >> choice;

        switch (choice) {
            case 1:
                q.addOrder();
                break;
            case 2:
                q.cancelOrder();
                break;
            case 3:
                q.orderReady();
                break;
            case 4:
                q.displayOrders();
                break;
            case 5:
                q.getFront();
                break;
            case 6:
                cout << "Exiting program.\n";
                return 0;
            default:
                cout << "Invalid choice. Please try again.\n";
        }
    }
}
```

3. Individual Contribution

3.1 Code Distribution

Task	Member involved
Synopsis and Objective	Tan Yi Ya
System Requirement & Analysis	Lam Yoke Yu
Class Diagram	Tan Yi Ya
`main()`	Tan Yi Ya
`addOrder()` and `getFront()`	Hafizi
`displayOrder()` and `orderReady()`	Miya
`cancelOrder()`	Lam Yoke Yu
Completing the code	Tan Yi Ya

3.2 Log Sheet

1. DisplayOrder() - prepared by Damiya Aina

   Purpose: Shows all the orders in the queue with their details (orderID & quantity)

   How it works:

   1. Traverses through the queue, starting from the first order (queueHead)
   2. It retrieves and prints the orderID and quantity
   3. Display message like “No order in the queue” if the queueHead is nullptr

2. OrderReady() - prepared by Damiya Aina

   Purpose: to notify and remove the front(first) order when the order is ready

   How it works:

   1. It checks if the queue is empty
   2. If there’s an order:
      - It retrieves and prints the details of the first order (queueHead)
      - It removes the front order from the queue

3. The next order (if any) becomes the new front

3. CancelOrder() - prepared by Lam Yoke Ku

Purpose: to delete order of the inputted orderID

How it works:

1. If the queue is not empty, the function will read an input from the user for the order ID to be deleted.
2. Traverses through the queue, starting from the first order (queueHead) and comparing the orderID with the user input.
3. If the order exists, the order is deleted and the queue is updated.

4. AddOrder() - prepared by Hafizi Syamir

Purpose: Responsible for adding a new order to the queue. This operation allows users or a system to dynamically add incoming orders to be processed in the order they arrive (First-In-First-Out, or FIFO).

How it works:

1. The function prompts the user to enter the Order ID and Order Quantity.
2. Dynamically allocates memory for a new Order object.
3. If the queue is empty, both queueHead and queueTail are nullptr.
4. If the queue is not empty, the current queueTail's next pointer is updated to point to the new order.

4. GetFront() - Prepared by Hafizi Syamir

Purpose: Used to retrieve and display the details of the first order in the queue without removing it.

How it works:

1. The function first checks if the queue is empty by verifying if queueHead is nullptr.
2. If the queue is not empty, it retrieves the front order (pointed to by queueHead) and displays its details.
3. If the queue is empty, it displays a message indicating that there are no orders in the queue.

3.3 Documentation Contributions

LINK DEMO VIDEO